SESSION GUARDIAN · CASE STUDY · WEEK ONE

Cold returns went from a fifth of usage to 3% in a week

Two weeks of one heavy Claude Code user's logs before Session Guardian, and the first week with it: what changed, which parts of the plugin did the work, and what one week can't show yet.

The cost it goes after

On a Claude subscription, the main conversation stays cached for **one hour** after the last message. While it is cached, every message re-reads the conversation at the cheap cache-read price. Come back after the hour and the next message writes the whole conversation into the cache again, at **twice the normal input price**.

That one message, a **cold return**, costs as much as about 80 ordinary messages on Fable 5.1 and about 40 on Opus 5.5. On a 700K session it came to $12 to $15 at API prices.

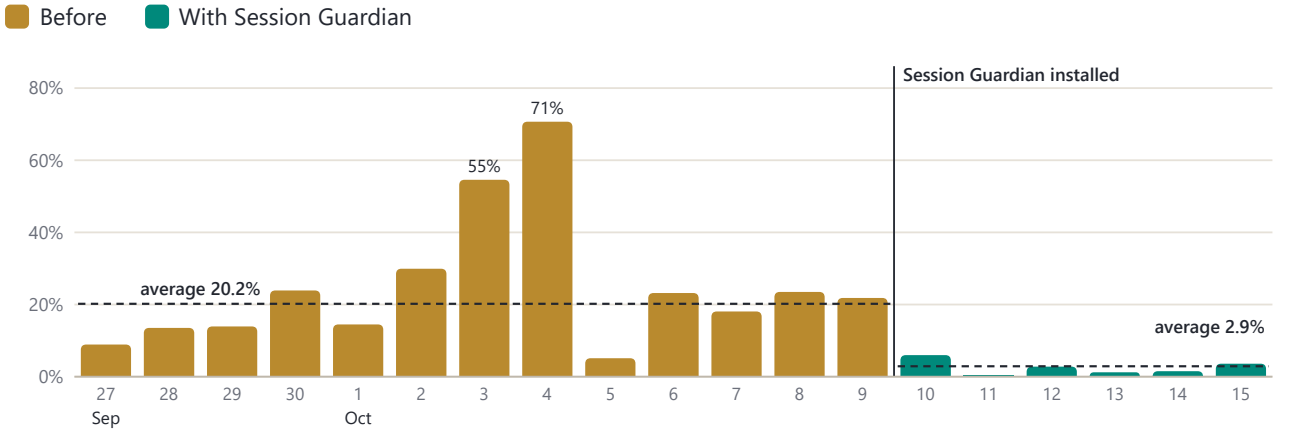
The week in four numbers

before 20.2%	before 13.7	before 402K	before 13%
2.9%	**2.5**	**264K**	**0%**
of usage went to cold returns	cold returns a day	context re-read by an average message	of sessions grew past 800K tokens

Before: 27 Sep to 9 Oct (13 days). With Session Guardian: 10 Oct to 16 Oct, 11:48 (6.5 days). Interactive sessions, main conversation, priced at Anthropic's API list prices of October 2026.

Share of each day's usage that went to cold returns

Interactive sessions. The plugin was installed in the first hour of 10 October.



3 and 4 Oct were a light weekend; most of their usage was cold returns to one large website session. 11 Oct had none.

What Session Guardian does

A Claude Code plugin built on function hooks (early access, tested on Claude Code 2.1.289), plus a few CLAUDE.md rules. This study covers version 0.1.0 (10 Oct) and 0.2.0 (15 Oct).

In the plugin

- **Countdown.** For sessions over 150K tokens, a line above the prompt counts down the last 15 minutes of the cache and says what coming back will cost. After the hour it turns red.
- **One pop-up** 5 minutes before the cache goes cold.
- **Wrap-up nudges** at 400K and again at 800K tokens.
- `/wrapup`: Claude saves where the session stopped, marks it done and gives a one-line prompt for the next session.
- **Retry guard.** A Bash or PowerShell command that failed twice in a row with no edit in between is paused once on its third try.
- **Dead ends** (0.2.0). Commands that kept failing are remembered with their error line and what worked after them, and `/wrapup` writes them into the notes.

In the rules and the repo

- **Numbered sessions.** Each session takes a number from `SESSIONS.md` and shows its state in the sidebar: `@` waiting on you, `✓` done, `!` blocked.
- `TRACKER.md`, one file a fresh session reads to pick up the work. Parallel sessions each edit only their own line.
- **TLDR on long replies** (optional), so the last message says where a session stands.
- `bench/cold-returns.py` prices your own transcripts on your machine and finds the cold returns. Since 11 Oct it also splits each one into what was rewritten and what was still cached.

How it was measured

- Every Claude Code transcript on the machine (`~/.claude/projects`), read with the same logic as `bench/cold-returns.py`. Each request of the main conversation is priced at API list prices. That is a guide to how the weekly limit drains, not a bill.
- A cold return is a request more than 60 minutes after the previous one in the same session that writes at least 10K tokens back into the cache.
- The second window ends on 16 Oct at 11:48, when version 0.3.0 came out. It adds an automatic save before the cache goes cold and will be measured on its own.
- Subagents have their own 5-minute cache and are left out. So is one automated session, a background loop that woke every 110 minutes and so came back cold almost every time. Waking it within the hour fixed that on 11 Oct, with no plugin involved.

Before and after

	BEFORE, 13 DAYS	WITH IT, 6.5 DAYS
Cold returns' share of usage	20.2%	**2.9%**
Cold returns per day	13.7	**2.5**
Spent on cold returns per day	$57	**$6**
Average cold return	$4.13, 330K rewritten	**$2.38, 224K**
Fresh sessions per day	11.9	**20.8**
First message of a fresh session, median	$0.31	**$0.36**
Average context of a warm message	402K	**264K**
Average warm message	$0.164	**$0.121**
Messages that carried 400K or more	46%	**15%**
Requests per day	1,360	**1,591**
Usage per day at API prices	$280	**$206**
Model mix, share of requests	Opus 5.5 75%, Fable 5.1 24%	**Opus 5.5 81%, Fable 5.1 19%**

More requests a day, less usage a day: the average request cost $0.13 against $0.21 before.

Which parts did the work

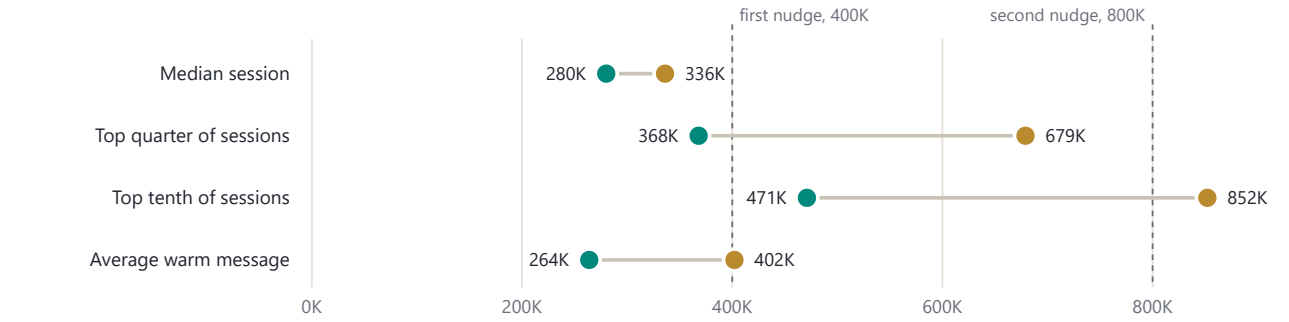
The features you run by hand barely ran. `/wrapup` was used once in the week, and the retry guard paused one command. Dead ends shipped on 15 Oct and have no data yet.

What changed was behaviour. With a countdown in view and a nudge at 400K, sessions were left to grow less and fresh sessions went from about 12 to 21 a day. Each fresh start cost a median $0.36, where the average cold return it replaced cost $4.13. Smaller sessions also make every ordinary message cheaper, since each one re-reads the whole context: the average warm message fell from $0.16 to $0.12.

Rough saving: about $51 a day less on cold returns and about $4.50 a day more on fresh starts, so about $46 a day, or $320 a week at API prices. Against a week that fills a Max 20x limit (about $1,770 for this user) that is around 18%, a bit more than a day of usage back each week. The first two days had suggested 15% to 20%.

How big sessions got

Largest context each interactive session reached, and the context an average warm message re-read. Tokens.



Why the countdown starts at 150K

From 433 cold returns between 10 Aug and 16 Oct: the rewrite is 98% of what a cold return costs, and about 40K of every prompt was still cached. Returns that rewrote under 200K were half of all cold returns and a quarter of their cost. Smaller sessions are cheap to bring back, so they get no countdown.

REWRITTEN	SHARE OF RETURNS	SHARE OF COST
Under 100K	21%	7%
100K to 200K	29%	19%
200K to 400K	30%	37%
400K to 600K	12%	19%
600K and more	8%	18%

What one week can't show

- It is one user on one machine, over 6.5 days.
- The week began the day after the weekly limit ran out (9 Oct), so 10 and 11 Oct were light.
- Knowing the logs were being measured changes habits by itself.
- The model mix moved a little toward Opus 5.5, and the work differs from week to week.
- Anthropic doesn't publish how subscription limits are counted, so API prices are a guide.

The benchmark gets re-run in a few weeks, and the automatic save in 0.3.0 gets its own before and after. To check your own logs, run `python bench/cold-returns.py --since 2026-09-01` once now and again a few weeks after installing.